

CAPACITY-BUILDING TRAINING

Image Analysis for Plant Phenotyping

A 5-Day Open-Source Intensive — Participant Guide

From raw image acquisition to 3D reconstruction: acquisition, segmentation, classical machine learning, deep learning, and 3D vision — built on an open-source-first toolchain, deliverable in-person or hybrid.

Live course site: jdetras.github.io/image-analysis-phenotyping-training

Duration: 5 days (~35 hours) · **Format:** in-person / hybrid · **Cohort:** 16–24 participants

Materials licensed CC BY 4.0 · Example code MIT

Contents

1. Course Overview
2. 5-Day Programme
3. Setup & Pre-Arrival Checklist
4. Datasets, References & Licensing

1 Course Overview

Who this course is for

Researchers, technicians, graduate students, and breeding-program staff who collect plant images — from flatbed scans and growth-chamber cameras to field drones — and want to turn them into quantitative, analysis-ready traits. The course assumes:

- **Basic Python literacy** — you can read a script, run a function, and edit variables. A pre-course primer notebook is provided for refreshers.
- **A scientific background in plant biology, agronomy, or breeding** — or a computational background working alongside plant scientists.
- **No prior computer-vision or deep-learning experience required.** We build it up from first principles.

Cohort size is tuned for 16–24 participants with one instructor and one teaching assistant, so everyone gets hands-on support during practicals.

Learning outcomes

By the end of the week, participants will be able to:

- **Acquire & manage** — design an imaging protocol, control for illumination and scale, and organise datasets with reproducible metadata.
- **Pre-process** — apply denoising, colour-space transforms, and calibration so downstream measurements are robust and comparable.
- **Segment** — separate plant from background using thresholding, classical, and learned segmentation — and know when each fails.
- **Quantify shape** — extract morphometric traits (area, length, convex hull, skeleton, colour indices) and validate them against ground truth.
- **Apply classical ML** — engineer features and train interpretable classifiers/regressors for trait prediction and quality control.
- **Train deep models** — fine-tune CNNs, detectors (YOLO), and promptable segmenters (SAM) for counting, detection, and pixel labelling.
- **Reconstruct in 3D** — build point clouds and meshes via photogrammetry and measure volumetric/architectural traits.
- **Work reproducibly** — package an analysis in a versioned environment with notebooks, so a colleague can re-run it end to end.

Programme at a glance

Seven topics, woven across five days, with a cross-cutting *imaging, data & practicals* thread running through every afternoon.

Day	Theme	Core topics	Primary tools
1	Acquisition & image fundamentals	Imaging modalities, colour spaces, calibration, data management	scikit-image, OpenCV, Fiji
2	Noise, segmentation & shape	Denoising, thresholding, morphology, morphometrics	scikit-image, PlantCV, napari
3	Classical machine learning	Feature engineering, classification, regression, pixel classifiers	scikit-learn, ilastik
4	Deep learning for phenotyping	CNNs, transfer learning, detection, promptable segmentation	PyTorch, YOLO, SAM
5	3D vision & capstone	Photogrammetry, point clouds, meshes, mini-project	COLMAP/Meshroom, Open3D, CloudCompare

Format & logistics

- **Daily rhythm** — mornings are concept + guided demo (~3 h); afternoons are hands-on practicals on your own data or provided datasets (~3.5 h), with TA support.
- **Delivery** — in-person computer lab or hybrid via video + shared notebooks. All compute can run locally or on free Google Colab GPUs.
- **Assessment** — daily practical checkpoints plus a Day-5 capstone applied to a real phenotyping question. Optional certificate on completion.

2 5-Day Programme

Each day runs roughly 09:00–16:30 with a morning lecture/demo block, a long lunch, and an afternoon practical. Times are indicative — adjust to your timezone and breaks.

Day 1 — Acquisition & image fundamentals

scikit-image · OpenCV · Fiji

Morning — concepts & demo

- **Why imaging for phenotyping:** traits, throughput, and the measurement chain from sensor to number.
- **Imaging modalities:** RGB, multispectral, thermal, chlorophyll fluorescence, depth — what each measures and its artefacts.
- **How a digital image works:** pixels, bit depth, channels, raw vs. compressed, EXIF metadata.
- **Colour spaces:** RGB → HSV, Lab, and excess-green (ExG) indices; why colour space choice makes or breaks segmentation.
- **Calibration:** colour cards, scale bars, flat-field correction, and fixing geometric distortion.

Afternoon — practical

- Load, inspect, and convert images with scikit-image and OpenCV; read pixel statistics and histograms.
- Build a tidy dataset folder with a metadata CSV (plant ID, date, treatment, scale).
- Open the same images in Fiji/ImageJ for interactive inspection and measurement.
- **Checkpoint:** convert a batch of raw images to a calibrated, scale-annotated working set.

Day 2 — Noise filtering, segmentation & shape

scikit-image · PlantCV · napari

Morning — concepts & demo

- **Noise & denoising:** Gaussian, median, bilateral, non-local means, and total-variation filters; when smoothing destroys signal.
- **Thresholding:** global (Otsu), adaptive/local, and multi-channel colour thresholding.
- **Morphological operations:** erosion, dilation, opening, closing, watershed for touching objects.
- **Shape & morphometrics:** area, perimeter, convex hull, solidity, eccentricity, skeleton length, leaf count.

Afternoon — practical

- Build a classical segmentation pipeline in PlantCV for a rosette or seedling image set.
- Interactively label and correct masks in napari.
- Extract a morphometric trait table and plot trait distributions across treatments.
- **Checkpoint:** a reproducible mask-to-traits pipeline with a QC visual overlay.

Day 3 — Classical machine learning

scikit-learn · ilastik

Morning — concepts & demo

- **From pixels to features:** colour, texture (GLCM, LBP), and shape descriptors as model inputs.
- **Supervised learning basics:** train/validation/test splits, leakage, cross-validation, class imbalance.
- **Models:** logistic regression, SVM, random forests, gradient boosting — strengths and interpretability.
- **Evaluation:** accuracy vs. precision/recall/F1, confusion matrices, regression error metrics, calibration.
- **Pixel classification:** random-forest segmentation as a bridge to deep learning.

Afternoon — practical

- Engineer features from Day-2 masks and train a scikit-learn classifier (e.g. healthy vs. stressed).
- Train an interactive pixel classifier in ilastik and compare it to threshold-based segmentation.
- Run cross-validation and read the confusion matrix critically.
- **Checkpoint:** a validated trait classifier with an honest performance report.

Day 4 — Deep learning for phenotyping

PyTorch · YOLO · SAM

Morning — concepts & demo

- **Neural network intuition:** convolutions, feature maps, loss, backprop, overfitting — visually, not just mathematically.
- **Transfer learning:** why fine-tuning a pretrained CNN beats training from scratch on small phenotyping datasets.
- **Task families:** classification, object detection & counting (YOLO), semantic/instance segmentation.
- **Foundation models:** the Segment Anything Model (SAM) for promptable, label-efficient segmentation.
- **Annotation & data:** how much labelled data you really need, and augmentation strategies.

Afternoon — practical (GPU or Colab)

- Fine-tune a pretrained CNN in PyTorch for a phenotyping classification task.
- Train/run a YOLO detector to count organs (e.g. spikes, fruits, leaves).
- Use SAM to generate segmentation masks with minimal manual labelling.
- **Checkpoint:** a trained model with metrics, plus inference on held-out images.

No GPU? The full afternoon runs on free Google Colab GPUs — see §3 Hardware requirements.

Day 5 — 3D vision & capstone

COLMAP/Meshroom · Open3D · CloudCompare

Morning — concepts & demo

- **3D phenotyping motivation:** traits you can't get from 2D — volume, canopy architecture, self-occlusion.
- **Photogrammetry / structure-from-motion:** from a turntable image set to a point cloud with COLMAP or Meshroom.
- **Point-cloud processing:** registration, downsampling, normals, meshing in Open3D.
- **Measurement & QC:** volume, height, leaf-area estimates and visual inspection in CloudCompare.
- **Field scale:** a brief look at drone orthomosaics and plot extraction in QGIS (commercial equivalents noted).

Afternoon — capstone

- Apply any part of the week's toolkit to a phenotyping question on your own or a provided dataset.
- Package the analysis as a reproducible notebook + environment.
- Short show-and-tell: each participant presents one result and one thing that surprised them.
- **Checkpoint:** capstone notebook submitted for the optional certificate.

Mapping to the seven topics

#	Topic	Where it lives
1	Image acquisition	Day 1
2	Noise filtering	Day 2 (morning)
3	Segmentation	Day 2–3 (classical → learned)
4	Shape analysis / morphometrics	Day 2 (afternoon)
5	Classical machine learning	Day 3
6	Deep learning	Day 4
7	3D vision	Day 5
—	Imaging, data & practicals (cross-cutting)	Every afternoon

Assessment & certification

- **Daily checkpoints (60%)** — one short practical deliverable per day, reviewed by the TA.
- **Capstone (40%)** — a reproducible notebook answering a phenotyping question, presented on Day 5.
- **Certificate of completion** issued to participants who attend $\geq 80\%$ and submit the capstone.

3 Setup & Pre-Arrival Checklist

Do this at least 3 days before Day 1. Installing on conference Wi-Fi on the morning of the course never ends well.

- Install [Miniforge](#) (conda + the conda-forge channel; includes the fast mamba solver).
- Download the course repository (`git clone` or "Download ZIP").
- Create the environment: `conda env create -f env/environment.yml`
- Activate it: `conda activate phenotyping`
- Run the verifier: `python notebooks/00_check_environment.py` — every required line should read `[ ok ]`.
- Install the GUI apps you'll use (Fiji, ilastik — see the table below).
- Create a free [Google Colab](#) account as the Day-4 GPU fallback.
- (Optional) Bring a small set of your own plant images to use in the practicals.

Install the Python environment

The repository ships a `conda environment.yml` for a one-command setup (use `mamba` in place of `conda` for a faster solve):

```
git clone https://github.com/jdetras/image-analysis-phenotyping-training.git
cd image-analysis-phenotyping-training
conda env create -f env/environment.yml
conda activate phenotyping
python notebooks/00_check_environment.py
```

NVIDIA GPU users: the environment installs the CPU build of PyTorch so it works everywhere. After creating the env, install the CUDA build matching your driver from pytorch.org/get-started/locally. No conda? A `requirements.txt` is provided for a plain `virtualenv` or `Colab` — always create a `virtualenv` first.

Software stack — Python / command-line

These install via the environment file above. Licenses summarised; confirm against each project before redistribution.

Tool	Role in the course	License	Source
scikit-image	Core image processing — filters, segmentation, morphometrics	BSD-3	scikit-image.org
OpenCV	Fast image I/O, transforms, classical CV ops	Apache-2.0	opencv.org
PlantCV	Plant-specific phenotyping pipelines & trait extraction	MIT	plantcv.org
scikit-learn	Classical ML — classifiers, regressors, cross-validation	BSD-3	scikit-learn.org

Tool	Role in the course	License	Source
PyTorch	Deep learning framework (CNNs, transfer learning)	BSD-3	pytorch.org
torchvision	Pretrained models, datasets, image transforms	BSD-3	pytorch.org/vision
Ultralytics (YOLO)	Object detection & counting of plant organs	AGPL-3.0	github.com/ultralytics
Segment Anything (SAM)	Promptable, label-efficient segmentation	Apache-2.0	github.com/facebookresearch
Open3D	Point-cloud & mesh processing for 3D phenotyping	MIT	open3d.org
JupyterLab	Notebook environment for all practicals	BSD-3	jupyter.org

Software stack — GUI applications

Install these manually from the official download pages. They cover interactive inspection, labelling, and 3D work where a point-and-click tool beats a script.

App	Role in the course	License	Download
Fiji / ImageJ	Interactive image inspection & measurement (Day 1–2)	GPL / PD	fiji.sc
napari	n-D image viewer & manual mask correction (Day 2)	BSD-3	napari.org
ilastik	Interactive random-forest pixel classification (Day 3)	GPL-2.0+	ilastik.org
CloudCompare	Point-cloud inspection & measurement (Day 5)	GPL	cloudcompare.org
Meshroom	GUI photogrammetry / structure-from-motion (Day 5)	MPL-2.0	alicevision.org
COLMAP	Structure-from-motion / multi-view stereo (Day 5)	BSD-3	colmap.github.io
QGIS	Drone orthomosaics & plot extraction (Day 5)	GPL-2.0+	qgis.org

Open-source-first: where teams already license commercial tools, the common equivalents are Pix4D / Agisoft Metashape (→ COLMAP/Meshroom + QGIS) and MATLAB + Image Processing Toolbox (→ scikit-image / OpenCV). These are noted but not used in practicals.

Apple Silicon & MLX (Mac alternative to PyTorch)

On Apple Silicon (M-series) Macs the whole stack installs and runs natively as arm64, and PyTorch automatically uses the Metal (MPS) GPU — the Day-4 practical works out of the box.

Optionally, add **MLX**, Apple's array/ML framework. On Apple Silicon it runs on the GPU via Metal with unified memory and is often faster than PyTorch-MPS for training small CNNs from scratch — a good alternative for the Day-4 classification task. YOLO and SAM stay on PyTorch (which uses MPS), so MLX is an alternative for the CNN-training portion, not a whole-stack replacement. Install it with:

```
./setup.sh --with-mlx      # offered automatically on Apple Silicon
pip install -r env/requirements-mac-mlx.txt  # into your activated env
```

Apple Silicon only — there are no MLX wheels for Intel Macs, and MLX has no Metal GPU inside Docker on a Mac, so use the native install. The verifier confirms it with a line like “[ok] MLX works (default device: Device(gpu, 0))”.

Hardware requirements

Component	Minimum	Recommended
CPU	4 cores	8+ cores
RAM	8 GB	16–32 GB (3D & large images)
Disk	20 GB free	50 GB free (datasets + models)
GPU	None — use Colab for Day 4	NVIDIA GPU ≥6 GB VRAM for local DL
OS	Windows 10+, macOS 12+, or Linux	Linux or macOS for smoothest 3D tooling

No-GPU path: Days 1–3 and the Day-5 3D work run comfortably on CPU. Only the Day-4 deep-learning practical benefits from a GPU — and that runs end-to-end on free Google Colab GPUs, so a modest laptop is enough to complete the whole course.

4 Datasets, References & Licensing

Datasets

Open phenotyping datasets used in demos and available for practicals. Always check each dataset's own license and citation requirements before reuse or redistribution.

Dataset	Used for	Source
CVPPP — leaf segmentation & counting	Day 2–4: segmentation, counting	plant-phenotyping.org/datasets
PlantVillage — leaf disease images	Day 3–4: classification	github.com/spMohanty/PlantVillage-Dataset
Global Wheat Head Detection (GWHD)	Day 4: detection & counting	global-wheat.com
PlantCV tutorial datasets	Day 2: classical pipelines	plantcv.readthedocs.io
Pheno4D — 4D plant point clouds	Day 5: 3D processing	ipb.uni-bonn.de/data/pheno4d
Your own images	Capstone	Bring a small set on Day 1

Bandwidth tip: hosts should mirror the chosen datasets to a local drive before the course — several are multi-GB and venue Wi-Fi rarely copes with 20 people downloading at once.

Key references

- Gehan et al. (2017). PlantCV v2: Image analysis software for high-throughput plant phenotyping. *PeerJ*. doi:10.7717/peerj.4088
- van der Walt et al. (2014). scikit-image: image processing in Python. *PeerJ*. doi:10.7717/peerj.453
- Kirillov et al. (2023). Segment Anything. *ICCV*. arXiv:2304.02643
- David et al. (2020). Global Wheat Head Detection (GWHD) dataset. *Plant Phenomics*. doi:10.34133/2020/3521852
- Minervini et al. (2016). Finely-grained annotated datasets for image-based plant phenotyping. *Pattern Recognition Letters*. doi:10.1016/j.patrec.2015.10.013
- Paulus (2019). Measuring crops in 3D: using geometry for plant phenotyping. *Plant Methods*. doi:10.1186/s13007-019-0490-0

Licensing

Course materials (text, slides, notebooks, and the website) are released under Creative Commons Attribution 4.0 (CC BY 4.0); the example code is additionally available under the MIT License. Every referenced tool retains its own license. Two points warrant attention for organisations:

- Ultralytics YOLO is AGPL-3.0.** Fine for training, research, and teaching, but using it inside a closed-source product can trigger copyleft obligations — evaluate before embedding it in proprietary pipelines.
- Fiji, ilastik, CloudCompare, QGIS** are GPL-family — free to use and share; distributed modifications must stay open. SAM, Open3D, PlantCV, scikit-image, scikit-learn, PyTorch, and OpenCV are permissive (Apache-

2.0 / BSD / MIT).

Not legal advice. License summaries are provided in good faith for planning; confirm current terms on each project's site before redistributing software or building commercial products.

Host / instructor checklist

- ☐ Test the full environment install on each target OS one week ahead.
- ☐ Pre-download model weights (YOLO, SAM) and datasets onto a shared drive as a Wi-Fi fallback.
- ☐ Confirm Colab access works on the venue network (some institutional firewalls block it).
- ☐ Share the pre-arrival checklist with participants 1–2 weeks ahead.
- ☐ Reserve a room with reliable power outlets at every seat.
- ☐ Recruit one TA per ~12 participants for afternoon practicals.